

# "R" programming

## UNIT-I

### 'R' - Introduction ①

- R is a programming language & environment used for statistical computing & representation.
- Originally developed by Ross Ihaka & Robert Gentleman & is managed by the 'R' Development core team.
- Today, Google, Facebook, Microsoft, Bank of America & many other companies use R.
- R is an open-source, interpreted computer language that contains more than 8000 packages used for statistical analysis, data manipulation, data visualization, ML & more. And a powerful solution for Big Data Analysis.

#### \* The main Advantages of using R :-

- It is open source & hence free.
- It has excellent functionality for effective graphical representation of data.
- It is easy to use.

#### \* Important features :-

- R has well-defined conditional loops, rich built-in fns, facility for user-defined recursive fns & effective I/O facilities.
- R allows the user to import data from a wide variety of resources, such as text files, statistical packages & diff. data stores.
- R provides a vast set of operators to work on different data types, such as arrays, lists, factors, frames, vectors & matrices.

\* R is licensed under the GNU General public ~~by~~ License (GPL) & is freely available for research & development.

### \* Installation

Download & Install R-tools & R-studio from the appropriate R-executables <sup>with proper</sup> versions.

### \* Installation And Loading of packages

→ we can install the required packages using the `install.packages()` function.

Ex: To install `ggplot2` package, that helps users to plot charts & graphs, we can use the following command in the R prompt

```
> install.packages("ggplot2")
```

→ packages can be installed from the R command line or from the shell.

```
> install.packages()
```

→ To use specific package, just load it in R using the `library()` function.

```
> library(ggplot2)
```

### \* R - Basic Syntax

we can enter the code for a program either at the R command prompt or in an R script file.

## \* R Command prompt

"R" is a case-sensitive language.

⇒ \$R

```
> tempstring ← "Hello, R world!"
```

```
> print(tempstring)
```

Op: "Hello, R World!"

→ first stmt assigns a string "Hello, R world!" to the string variable - tempstring.

→ The print() fn: on the next line prints the value stored in tempstring.

## \* R script file

→ While writing a ppgm, we type all the stmts in a file, called the Rscript.

→ we can execute these scripts at the command prompt with an R interpreter.

→ The same sample code written above is saved in a file called test.R

```
tempstring ← "Hello, R world!"
print(tempstring)
```

It can be executed as

```
> Rscript test.R
```

Op: "Hello, R world!"

→ The source("filename") fn: submits a script file to the current session of R. Then we can run ~~you~~ one Rscript.

```
> source("test.R")
```

Op: "Hello, R world!"

## \* Data types And Objects



## \* Variables

variables are used to store any type of data whose value can be changed. An Identifier is a unique name given to a variable.

→ Rules to create Identifiers are :-

- Identifiers can be a combination of letters, digits, period (.) & underscore (\_)
- They must start with<sup>a</sup> letter or a period (.)
- Reserved words cannot be used.
- They cannot ~~the~~ start with a ~~space~~ digit.

\* Constants : constants are entities whose value cannot be changed during the execution of the program.

→ constants are of two types :- Numeric & character.

⇒ Numeric Constants : Numeric constants can take any numeric value of the type Integer, double or complex.

- The type of the constant can be verified using the typeof() function.

\* If we want to assign an integer value to a constant, it should be followed by L;

& if we want to assign complex<sup>type</sup> value to a constant, it should be followed by i.

⇒ The default<sup>data</sup> type is double.

Ex > type of (8)  
double  
> type of (8L)  
integer  
> type of (8i)  
complex

\* Character constants :- character const's can be represented using single quotes (' ') or double quotes (" ") as delimiters.

\* Built-in constants :- Some built-in constants have been defined in R along with their values

Ex : > LETTERS # set of all capital alphabets

> letters # set of all small alphabets

> pi # value of Pi

np: 3.141593

> month.name

\* Debugging in R : when a program has no syntax error but the o/p is incorrect due to a logical error, we need to find the error or bug in the Pgm.

→ The procedure to identify & fix pgm bugs is called Debugging.

→ There are a number of functions that help in debugging programs:-

- `traceback()`
  - `debug()`
  - `browser()`
  - `trace()`
  - `reconer()`
- } are also used.

### \* Major players <sup>in</sup> Big data

- |                 |                    |
|-----------------|--------------------|
| - Google        | - Apache Mahout    |
| - Apache Hadoop | - Apache HBase     |
| - MapReduce     | - Apache Cassandra |

### \* Data Categorisation

Data can be categorised according to the scale of measure - most data has its own property and can be continuous or discrete.

\* Entity: A particular thing is called an entity or object. An entity can be a single thing, person, place or object.

- Data can be stored about such entities.

\* Attribute: An attribute is a measurable or observable property of an entity.

\* Data: A measure of an attribute of an entity is called data.

\* Scales of Measurement :- Scales of measurement describe the nature of the information assigned to a variable.

\* The four scales of measurement are nominal, ordinal, interval & ratio.  
quantitative data qualitative data

→ qualitative data comprises measures of values & is expressed as numbers or categories. It is descriptive & conceptual.

→ quantitative data is measured using numbers & values. It is concise & close ended. It is continuous & discrete.

\* Data Cube : A data cube is a multidimensional data model that views data in the form of a cube.

⇒ It is characterised by 2 parameters :-

dimension - <sup>which is the</sup> perspective or entity w.r.t which an organization wants to maintain a record.

fact - which comprises the actual values in the record

\* Data cube operations

- Roll-up
- Drill down
- Slice & dice
- Pivot

} data processing operations.



- \* Roll-up - <sup>this</sup> operation calculates the aggregation on a data cube based on rows or columns (or) hierarchy (or) whole data. (done by grouping based on a concept hierarchy (or) by reducing a dimension)
- \* Drill-down - Reverse of roll-up operation. Here, group data is expanded into its subparts. It can be performed by stepping down (expanding) a concept hierarchy.
- \* Slice :- In this, cross-tabulation is carried out for specific values other than for a fixed third dimension.
- \* Dice :- It defines a sub-cube by performing a selection on two or more dimensions.
- \* Pivot :- A technique of changing from one dimensional orientation to another, also called as rotation.

## \* Operators :-

Following operators are available in the 'R' language

- Arithmetic  $\rightarrow + - * / \% / \% \wedge$
- Relational  $\rightarrow < > == <= >= !=$
- Logical  $\rightarrow \& | ! \&\& ||$
- Miscellaneous  $\rightarrow : \%in\% \%*\%$
- Assignment  $\rightarrow = <- \rightarrow <<- \rightarrow$

$\rightarrow$  :  $\rightarrow$  colon operator, used to create a sequence of nos <sup>for</sup> a vector.

$\%in\%$   $\rightarrow$  used to identify if an element belongs to a vector.

$\%*\%$   $\rightarrow$  used to multiply matrices.

\* Control Statements : Control stmts enable you to control the flow of execution inside a function.

→ commonly used control stmts are: if, else, for, while, repeat, break, next & return.

\* The if statement :

"if" contains a condition, if the condition evaluates as TRUE, the R code associated with the if statement is evaluated

```
if (cond)
{
  # DO
}
```

→ if-else : The code associated with the else statement is executed whenever the "if" cond is not satisfied.

```
if (cond)
{
  # DO
}
else
{
  # DO else
}
```

\* The for statement :

→ The for, while & do-while statements are used to iterate through the written code.

→ The two kinds of loops here are - Entry controlled & Exit controlled.

(6)

→ In entry controlled loops, the test condition is checked first & if it is false, the loop body will not be executed at all. ~~can~~

→ In the case of exit controlled loops, the loop body will be executed ~~once~~ ~~one~~ ~~of~~ even if the test condition is false.

→ for & while come under entry controlled category  
The do-while loop is an exit controlled loop.

Syntax :

for (Variable in start : end)

{ stmts

}

\* while loop :-

syntax : while (test-expression)

{ stmts

}

Here, the test-expression evaluates to Boolean value. statements inside the while loop will be executed only if test-exp evaluates to true.

→ so when we use the while loop, ensure that there is a way to exit the loop.

## \* The repeat And break stmts :

→ The repeat stmt executes the specified stmts repeatedly till the terminating cond. evaluates to True, after which execution breaks.

Syntax:

```
> repeat
{
  stmts
  if (test-expr)
  {
    break
  }
}
```

Ex > x <- 11

```
> repeat
{
  print(x)
  x = x + 1
  if (x <= 16)
  {
    break
  }
}
```

o/p: 11  
12  
13  
14  
15

}

## \* The next statement

~~There~~ If we want to skip the current iteration of a loop without terminating it.

→ when "next" stmt occurs, the R parser skips execution of the stmts for evaluation in the current iteration & instead starts the next iteration of the loop from the beginning.

Ex > for (t in 1:20)

```
{
  if (t % 2 == 1)
  {
    next
  }
  else
  {
    print(t)
  }
}
```

o/p: 2  
4  
6  
8  
10  
12  
14  
16  
18  
20

## \* The switch statement

syntax: switch (exp-variable, case 1, case 2, case 3 ---)

→ Here ~~the~~ we can check expression-variable against a list of values specified as case 1, case 2, ....

→ The parameter exp-variable is an integer value.

Ex     $x \leftarrow \text{switch}(2, \text{"one"}, \text{"two"}, \text{"three"})$

$> x$

Op: "two".

## \* Functions:

→ R fn's are very similar to the fn's of C, Java ---

→ R fn's are like R objects. i.e we can perform everything with fn's that ~~you~~<sup>we</sup> would do with 'R' objects.

Syntax : function-name  $\leftarrow$  function(arguments)

{ function body

----

}

\* return() fn: :- The return() fn: returns the result of the processing performed by a fn:

syntax : return(expression)

Any valid object can be returned from the return stmts.

→ The value of the last evaluated expression is returned automatically in the case of missing returns from a fn:

→ The return stmts should be the last stmts of the fn:

if not, the fn: will end permanently & control will go back to the fn: call.



# Expressions, Variables & Functions

## i) Expressions

Few arithmetic operations : Addition, subtraction, multiplication, division, exponentiation, finding remainder (modulus), integer division & computing square root.

| operation               | operator                 | Example                      |
|-------------------------|--------------------------|------------------------------|
| → Addition              | $x + y$                  | $> 4 + 8$<br>[1] 12          |
| → subtraction           | $x - y$                  | $> 10 - 3$<br>[1] 7          |
| → multiplication        | $x * y$                  | $> 7 * 8$<br>[1] 56          |
| → Division              | $x / y$                  | $> 8 / 3$<br>[1] 2.666       |
| → Exponentiation        | $x \wedge y$<br>$x ** y$ | $> 2^5$<br>[1] 32            |
| → modulus               | $x \% y$                 | $> 5 \% 3$<br>[1] 2          |
| → Integer division      | $x \% / \% y$            | $> 5 \% / \% 2$<br>[1] 2     |
| → Computing square root | $\text{sqrt}(x)$         | $> \text{sqrt}(25)$<br>[1] 5 |

## ii) Logical values

'R' is case sensitive  
Rmg

Logical values are TRUE & FALSE

(or) T & F

Equality operator is ==

Ex > 8 < 4

[1] FALSE

> 3 \* 2 == 6

[1] TRUE

> F == FALSE

[1] TRUE

## iii) Dates

The default format of date is YYYY-MM-DD

i) Print system's date

> Sys.Date()

[1] "2017-01-13"

ii) print system's time.

> Sys.time()

[1] "2017-01-13 10:54:37 IST"

iii) print time zone

> Sys.timezone()

[1] "Asia/calcutta"

iv) print today's date

> today <- Sys.Date()

> today

[1] "2017-01-13"

> format(today, format = "%B %d %Y")

[1] "January 13 2017"

v) store date as a text data type

> custodate = "2016-01-13"

> custodate

[1] "2016-01-13"

> class(custodate)

[1] "character"

vi) Convert the date stored as text data type into a date data type.

> custdate = as.Date(custodate)

> class(custdate)

[1] "Date"

> custdate

[1] "2016-01-13"

vii) Find the difference between the following two dates

> strdates <- c("08/15/1947", "01/26/1950")

viii) Convert strings into date format

> dates = as.Date(strdates, "%m/%d/%Y")

> dates

[1] "1947-08-15" "1950-01-26"

ix) > dates[2] - dates[1]

## IV) Variables

i) Assign a value of 50 to the variable called 'var'

```
> var <- 50
```

(or)

```
> var = 50
```

ii) print the value in the variable 'var'

```
> var ##
```

```
[1] 50
```

iii) perform arithmetic operations on the variable 'var'

```
> var + 10
```

```
[1] 60
```

```
> var / 2
```

```
[1] 25
```

\* variables can be <sup>re-</sup>assigned values either of the same data type or of a different data type.

```
> var <- TRUE
```

```
> var
```

```
[1] TRUE
```

```
> class(var)
```

```
[1] logical
```

```
> var <- "R programming"
```

```
> var
```

```
[1] "R programming"
```

## v) Functions

we have default function's already predefined

Some of them are `sum()`, `min()`, `max()` & `seq()`

→ `sum()` fn: → returns the sum of all the values in its arguments

Syntax: `sum(..., na.rm = FALSE)`

↳ complex (or) numeric (or) logical vectors

`na.rm` → accepts a logical value, where missing values (include `NaN` (Not a Number)) be removed?

Ex > `sum(1, 2, 3)`

[1] 6

> `sum(1, 5, NA, na.rm = FALSE)`

[1] NA

if `na.rm = TRUE`

> `sum(1, 5, NA, na.rm = TRUE)`

[1] 6

if `FALSE`, an `NA` or `NaN` value in any of the arg. will cause `NA` or `NaN` to be returned.

→ `min()` fn → returns the minimum of all the values present in their arguments.

if `na.rm = FALSE`, an `NA` or `NaN` value in any argument will cause `NA` or `NaN` to be returned.

→ `max()` fn → returns the maximum of all the values present in their arguments.



## vi) Manipulating Text in Data

There are many inbuilt string fns available in R that manipulate text or string.

- finding a part of some text string : `substr(a, start, stop)`
  - searching some string in a text : `grep(pattern, a)`
  - concatenating strings : `paste(..., sep = "...", ...)`  
ex `paste(a, b)`
  - convert the strings : `toupper(a)`  
`tolower(a)`
  - split the strings : `strsplit(a, split, ...)`
- a = "\_\_\_\_\_"  
↓  
Searches for pattern in a & gives Index position of pattern in given string

## \* Missing values Treatment in R

- During analytical data processing, we come across problems caused by missing & infinite values.
- To get accurate op, users should remove or clean the missing values.
- In R, `NA` → Not Available  
`NaN` → Not a Null  
`Inf` → Infinite value.

Ex: `x <- c(25, 86, 9, NA, 45, 3)`  
`y <- c("red", NA, "NA")`

## \* Finding the missing values

→ `is.na()` \* This fn: is used in R to identify the missing values. It returns a boolean value either TRUE or FALSE

```
> is.na(x)
```

```
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE
```

```
> is.na(y)
```

```
[1] FALSE TRUE FALSE
```

This `is.na()` fn: is used to find & create missing values.

→ `na.action()` provides options for treating the missing data.

↳ This include `na.omit()`  
`na.exclude()` } → These fn: returns the objects ~~unchanged~~ by removing the missing values.

→ `na.pass()` - returns object unchanged even with missing ~~to~~ objects.

→ `na.fail()` - returns object if it has no missing values.

ex `> c <- as.data.frame (matrix c(1:5, NA), ncol=2)`

```
> c
```

```
[1]    v1    v2
     1    1    4
     2    2    5
     3    3   NA
```

```
> na.omit(c)
```

```
[1]    v1    v2
     1    1    4
     2    2    5
```

> na.exclude(c)

|   | $v_1$ | $v_2$ |
|---|-------|-------|
| 1 | 1     | 4     |
| 2 | 2     | 5     |

> na.pas(c)

|   | $v_1$ | $v_2$ |
|---|-------|-------|
| 1 | 1     | 4     |
| 2 | 2     | 5     |
| 3 | 3     | NA    |

> na.fail(c)

[1] error in na.fail().default(c) : missing value in object

### \* Invalid values and Outliers

In R, special checks are conducted for handling invalid values.

→ An invalid value can be NA, NaN, Inf or -Inf.

• fn's for these invalid values are :-

→ anyNA(x) - returns TRUE val. if the ilp has any NA or NaN values.  
Equivalent to any(is.na(x))

→ anyInvalid(x) - returns TRUE if the ilp has any invalid value.  
Equivalent to any(is.valid(x))  
is.invalid(x)

(where value of x can be a vector, matrix or array.)  
→ returns an object corresponding to each ilp value.

if ilp is invalid returns TRUE  
else FALSE.

Ex : > anyNA (c(-9, NaN, 9))

[1] TRUE

> is.finite (c(-9, Inf, 9))

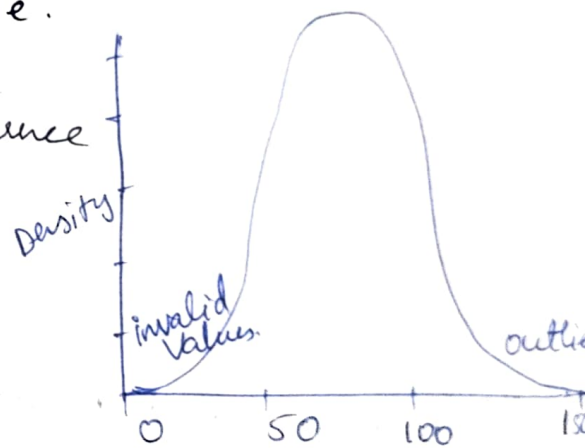
[1] TRUE FALSE TRUE

> is.nan (c(-9, Inf, 9))

[1] FALSE FALSE FALSE

→ The basic idea of invalid values & outliers can be explained with a simple example.

⇒ fig. helps in understanding difference b/w invalid values & outliers.



Ex > Summary (custdata \$ income)

| min   | 1 <sup>st</sup> Quantile | median | mean  | 3 <sup>rd</sup> Qu | Maximum |
|-------|--------------------------|--------|-------|--------------------|---------|
| -8700 | 14600                    | 35000  | 53500 | 67000              | 615000  |

> Summary (custdata \$ age)

| min | 1 <sup>st</sup> Qu | median | mean | 3 <sup>rd</sup> Qu | Maximum |
|-----|--------------------|--------|------|--------------------|---------|
| 0.0 | 38.0               | 50.0   | 51.7 | 64.0               | 146.7   |

→ The above two scenarios clearly explain the invalid & outlier values.

→ In the first scenario, ~~the~~ one of the values of 'income' is negative (-8700). practically a person cannot have negative income.

↳ Indicates debt.

However negative values required to be treated.

Either we need to drop the values for income or convert them into zero. (This should be handled properly)

→ In the second case, one of the values of 'age' is zero & other value is greater than 120 which is considered as an outlier. (i.e. values fall out of the data range of the expected values.

→ Here age=0 could refer to unknown data & age>120 may be little unusual data.

On this case, we need to check whether to drop the field, drop the data if or convert the improper data.

## \* Descriptive Statistics

- Data Range
- Frequencies & mode
- Mean & median
- standard deviation
- mode



i) Data Range : It helps in identifying the extent of difference in input data. The data range of the observation variable is the difference b/w the largest and the smallest data value in a dataset.

i.e.  $\text{Range} = \text{Largest value} - \text{Smallest value}$ .

for ex: # calculates the duration of rainfall  
 $> \text{duration} = \text{time} \$ \text{rainfall}$ .

# Apply max & min function to return the range  
 $> \text{max}(\text{duration}) - \text{min}(\text{duration})$

The above code returns the range of duration by taking the minimum & maximum values. In the above example, time duration of rainfall is helpful in predicting the probability of duration of rainfall.

ii) Frequencies and Mode

→ Frequency is a summary of data occurrence in a collection of non overlapping types.

→ In R, freq function can be used to find the frequency distribution of vector inputs.

for ex:  $> \text{head}(\text{subset}(\text{mtcars}, \text{select} = 'gear'))$

|                   | gear |
|-------------------|------|
| mazda RX4         | 4    |
| mazda RX4 Wag     | 4    |
| Datsun 710        | 4    |
| Hornet 4 Drive    | 3    |
| Hornet Sportabout | 3    |
| valiant           | 3    |

```
> factor(mtcars$gear)
```

```
[1] 4 4 4 3 3 3 4 4 4 3 3 3 3 3 4 4 4 5 5 5 4 4 4 3
```

```
> w = table(mtcars$gear)
```

```
> w
```

|    |    |   |
|----|----|---|
| 3  | 4  | 5 |
| 15 | 12 | 5 |

```
> t = as.data.frame(w)
```

```
> t
```

|   | var1 | Freq |
|---|------|------|
| 1 | 3    | 15   |
| 2 | 4    | 12   |
| 3 | 5    | 5    |

```
> names(t) [1] = 'gear'
```

```
> t
```

|   | gear | Freq |
|---|------|------|
| 1 | 3    | 15   |
| 2 | 4    | 12   |
| 3 | 5    | 5    |

## ⇒ Mode

- Mode is similar to frequency, except that the value of mode returns the highest number of occurrences in a dataset.
- Mode can take both numeric and character as input data.
- Mode does not have any standard inbuilt fn: to calculate mode of the given inputs.

- Hence, a user defined function is required to calculate mode in R. Here i/p is a vector value & the o/p is the mode value.

for ex:

```
# create the fn:
getmode <- function(y)
{
  uniqy <- unique(y)
  uniqy [which.max(tabulate(match(y, uniqy)))]
}

# Define the i/p vector values
V <- c(5, 6, 4, 8, 5, 7, 4, 6, 5, 8, 3, 2, 1)

# Cal. the mode with user defined fn's
resultmode <- getmode(V)
print(resultmode)
```

~~# Define characters~~

O/P: [1] 5

### iii) Mean and Median

⇒ Mean: Mean is the sum of i/p values divided by the sum of the no. of inputs. It is also called the average of the i/p values.

fn: mean() gives the o/p of mean value in R.

Syntax: mean(x, trim=0, na.rm=FALSE, ...) <sup>default value.</sup>  
 where  $\downarrow$  input vector  $\rightarrow$  removes the missing values in the i/p vector if na.rm=TRUE

trim() specifies some drop in observations from both the sorted ends of the i/p vector

Ex : # Define a vector

$x \leftarrow c(15, 54, 6, 5, 9.2, 36, 5.3, 8, -7, -5)$

# Find ~~to~~ the mean of the vector i/p's

result.mean  $\leftarrow$  mean(x)

Print(result.mean)

Ex o/p: [1] 12.65

when  
→ ~~first~~ trim parameter is selected, it sorts the vector values first & drops the i/p values for calculating the mean based on the trim value from both the ends.

→ ex if trim = 0.4, 4 values from both ends of sorted vector are dropped.

i.e. (15, 54, 6, 5, 9.2, 36, 5.3, 8, -7, -5)

sorted to

(-7, -5, 5, 5.3, 6, 8, 9.2, 15, 36, 54)

& 4 values are removed from both ends

Ex  $x \leftarrow c(15, 54, 6, 5, 9.2, 36, 5.3, 8, -7, -5)$

# Find the mean of the vector i/p's

result.mean  $\leftarrow$  mean(x, trim = 0.3)

Print(result.mean)

o/p: 7.125

→ if na.rm = TRUE is used to remove the NA values from list before calculating the mean.



## 1.4 HANDLING PACKAGES IN R

A package in R is the fundamental unit of shareable code. It is a collection of the following elements:

- Functions
- Data sets
- Compiled code
- Documentation for the package and for the functions inside
- Tests – few tests to check if everything works as it should.

The directory where packages are stored is called a library. R comes with a standard set of packages. Others are available for download and installation as per requirement. As on date, there are over 10,000 plus packages available in CRAN. This is also one of the reasons behind the huge popularity and success of R.

Packages are used to share codes with others. One can develop their own R package. Any R user can then download, install and learn to use the package. Packages, therefore allow for an easy, transparent and cross-platform extension of the R base system.

R is an open source language; thus, new packages are being developed and updated by developers daily. Some of these packages may not work properly or may have bugs. Hence, it is not a good idea to use every new and updated package on R development environment. This can affect the stability of the development environment. A stable environment requires the *sandboxing technique* (a security mechanism often used to execute untested or untrusted programs or code from unverified or untrusted third parties, users, etc., without damaging/maligning the host machine or operating system or production environment) to test new packages or update a package before installing it in the development environment.

In general, there is a single package library with each installation of R on a computer. Users can change the path to that library to install a package on a different location other than the default package library. The command `.libPaths()` can be used to get or set the path of the package library.



**TABLE 1.2** Commonly used R packages for different purposes

| <i>Data Management</i>            | <i>Data Visualisation</i>           | <i>Data Products</i>                 | <i>Data Modelling and Simulation</i>                     |
|-----------------------------------|-------------------------------------|--------------------------------------|----------------------------------------------------------|
| dplyr, tidyr, foreign, haven etc. | ggplot, ggvis, lattice, igraph etc. | shiny, slidify, knitr, markdown etc. | MASS, forecast, bootstrap, broom, nlme, ROCR, party etc. |

| <i>Author(s)</i>                                   | <i>Package Name</i> | <i>Description</i>                                                                                                         | <i>Available At</i>                                                                                                             |
|----------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Gabor Csardi                                       | igraph              | It contains routines for network analysis and making simple graphs to represent social networks.                           | <a href="http://igraph.sourceforge.net/">http://igraph.sourceforge.net/</a>                                                     |
| Hadley Wickham                                     | ggplot              | It contains a set of grammar rules for implementing graphics in R. The package is used for creating high-quality graphics. | <a href="http://cran.r-project.org/web/packages/glmnet/index.html">http://cran.r-project.org/web/packages/glmnet/index.html</a> |
| Hadley Wickham                                     | lubridate           | The package provides functions to use dates in R in an easier way.                                                         | <a href="https://github.com/hadley/lubridate">https://github.com/hadley/lubridate</a>                                           |
| Hadley Wickham                                     | reshape             | It contains a set of tools for manipulation, aggregation and management of data in R.                                      | <a href="http://had.co.nz/plyr/">http://had.co.nz/plyr/</a>                                                                     |
| Ingo Feinerer                                      | tm                  | It contains functions to perform text mining in R. Text mining helps to work with unstructured data.                       | <a href="http://www.spatstat.org/spatstat/">http://www.spatstat.org/spatstat/</a>                                               |
| Jerome Friedman, Trevor Hastie, and Rob Tibshirani | glmnet              | It helps to work with the elastic-net and also regularised and generalised linear models.                                  | <a href="http://had.co.nz/ggplot2/">http://had.co.nz/ggplot2/</a>                                                               |

### 1.4.1 Installing an R Package

R comes with some standard packages that are installed when a user first installs R and additional packages can be installed separately. Users need to navigate through the package library and install a package in the desired location. Following commands are used for navigating through R package library and installing R package.

1. To start R, follow either Step 2 or 3. The assumption is that R is already installed on your machine.
2. If there is an "R" icon on the desktop of the computer that you are using, double click on the "R" icon to start R. If there is no "R" icon on the desktop then click on the "Start" button at the bottom left of your computer screen, and then choose "All programs", and start R by selecting "R" (or R X.X.X, where X.X.X gives the version of R, e.g. R 2.10.0) from the menu of programs.
3. The R console should show up.
4. Once you have started R, you can install an R package (e.g. the "ggplot2" package) by choosing "Install package(s)" from the "Packages" menu at the top of the R console. This will ask you for the website that you wish to download the package from. You can choose "Iceland" (or another country, if you prefer). It will also bring up a list of available packages that you can install, and you can choose the package that you want to install from that list (e.g. "ggplot2").
5. This will install the "ggplot2" package.
6. The "ggplot2" package is now installed. Whenever you want to use the "ggplot2" package after this, after having successfully started R, you first have to load the package by typing into the R console: `library("ggplot2")`.
7. You can get help on a package by typing the following at the R prompt: `help(package = "ggplot2")`

## **2.3 DATA TYPES IN R**

R is a programming language. Like other programming languages, R also makes use of variables to store varied information. This means that when variables are created, locations are reserved in the computer's memory to hold the related values. The number of



locations or size of memory reserved is determined by the data type of the variables. Data type essentially means the kind of value which can be stored, such as boolean, numbers, characters, etc. In R, however, variables are not declared as data types. Variables in R are used to store some R objects and the data type of the R object becomes the data type of the variable. The most popular (based on usage) R objects are:

- Vector
- List
- Matrix
- Array
- Factor
- Data Frames

A vector is the simplest of all R objects. It has varied data types. All other R objects are based on these atomic vectors. The most commonly used data types are listed as follows:

Data types supported by R are:

- Logical
- Numeric
  - ♦ Integer
- Character
- Double
- Complex
- Raw

`class()` function can be used to reveal the data type. Other R objects such as list, matrix, array, factor and data frames are discussed in detail in Chapter 3.

## **Logical**

TRUE / T and FALSE / F are logical values.

```
> TRUE
[1] TRUE
> class(TRUE)
[1] "logical"

> T
[1] TRUE
> class(T)
[1] "logical"

> FALSE
[1] FALSE
> class(FALSE)
[1] "logical"

> F
[1] FALSE
> class(F)
[1] "logical"
```

## 2.4 FEW COMMANDS FOR DATA EXPLORATION

This section will use functions such as `summary()`, `str()`, `head()`, `tail()`, `view()`, `edit()`, etc., to explore a dataset. The dataset used in this section is “mtcars” from the “datasets” package.

*Background to the mtcars dataset from R documentation:*



## 3.13 METHODS FOR READING DATA

R supports different types of data formats related to a database. With the help of import and export utility of R, any type of data can be imported and exported into R. In this section, you will learn about the different methods used for reading data.

### 3.13.1 CSV and Spreadsheets

Comma separated value (CSV) files and spreadsheets are used for storing small size data. R has an inbuilt function facility through which analysts can read both types of files.

#### *Reading CSV Files*

A CSV file uses .csv extension and stores data in a table structure format in any plain text. The following function reads data from a CSV file:

```
read.csv('filename')
```

where,

filename is the name of the CSV file that needs to be imported.

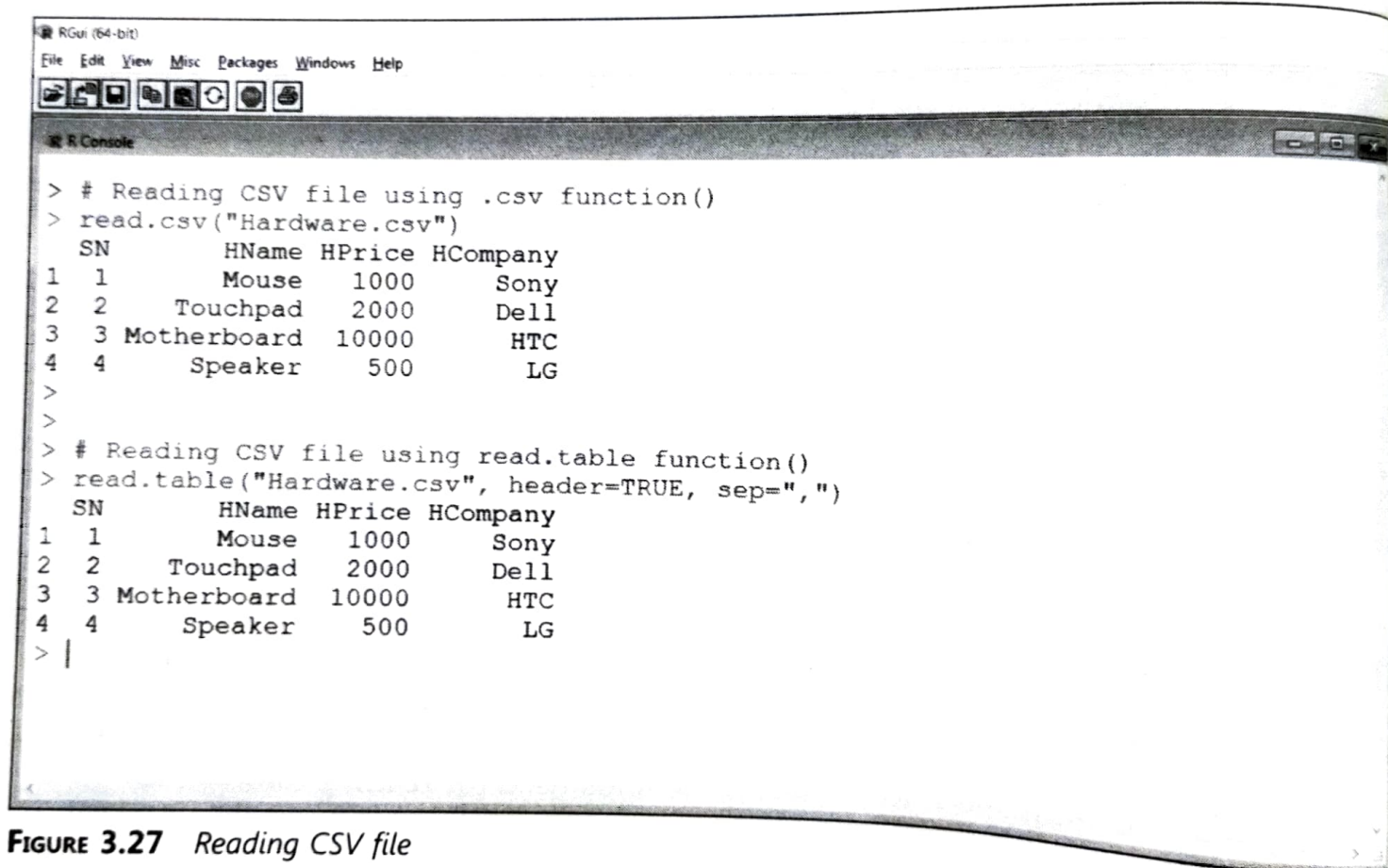
The `read.table()` function can also read data from CSV files. The syntax of the function is

```
read.table('filename', header=TRUE, sep=',', ...)
```

where,

**filename** argument defines the path of the file to be read, **header** argument contains logical values TRUE and FALSE for defining whether the file has header names on the first line or not, **sep** argument defines the character used for separating each column of the file and the dots '...' define the other optional arguments.

The following example reads a CSV file, 'Hardware.csv' using `read.csv()` and `read.table()` function (Figure 3.27).



```
> # Reading CSV file using .csv function()
> read.csv("Hardware.csv")
  SN      HName HPrice HCompany
1  1      Mouse   1000     Sony
2  2  Touchpad   2000      Dell
3  3 Motherboard 10000      HTC
4  4   Speaker    500       LG
>
>
> # Reading CSV file using read.table function()
> read.table("Hardware.csv", header=TRUE, sep=",")
  SN      HName HPrice HCompany
1  1      Mouse   1000     Sony
2  2  Touchpad   2000      Dell
3  3 Motherboard 10000      HTC
4  4   Speaker    500       LG
> |
```

FIGURE 3.27 Reading CSV file

## Reading Spreadsheets

A spreadsheet is a table that stores data in rows and columns. Many applications are available for creating a spreadsheet. Microsoft Excel is the most popular for creating an Excel file. An Excel file uses .xlsx extension and stores data in a spreadsheet.

In R, different packages are available such as `gdata`, `xlsx`, etc., that provide functions for reading Excel files. Importing such packages is necessary before using any inbuilt function of any package. The `read.xlsx()` is an inbuilt function of 'xlsx' package for reading Excel files. The syntax of the `read.xlsx()` function is

```
read.xlsx('filename', ...)
```

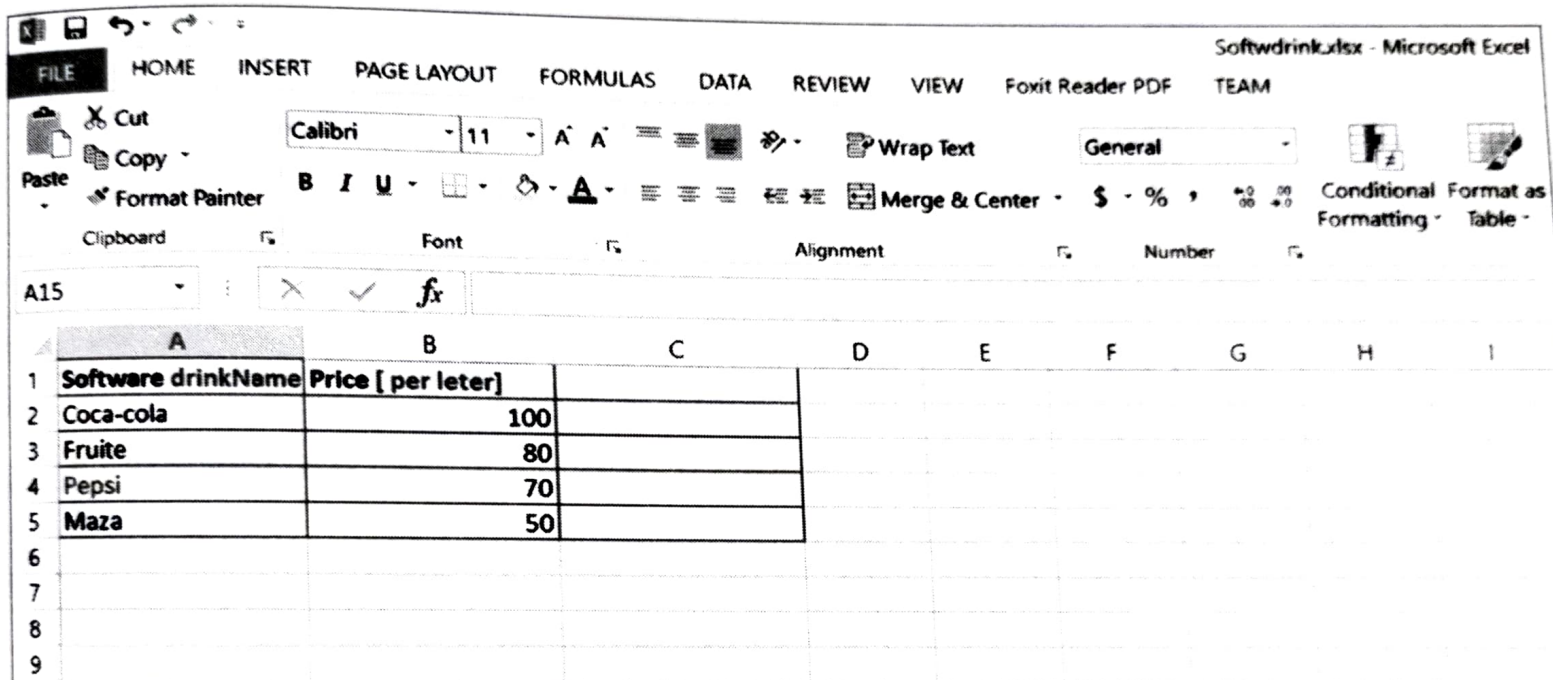


where,

filename argument defines the path of the file to be read and the dots '...' define the other optional arguments.

In R, reading or writing (importing and exporting) data using packages may create some problems like incompatibility of versions, additional packages not loaded and so on. In order to avoid these problems, it is better to convert files into CSV files. After converting files into CSV files, the converted file can be read using the `read.csv()` function.

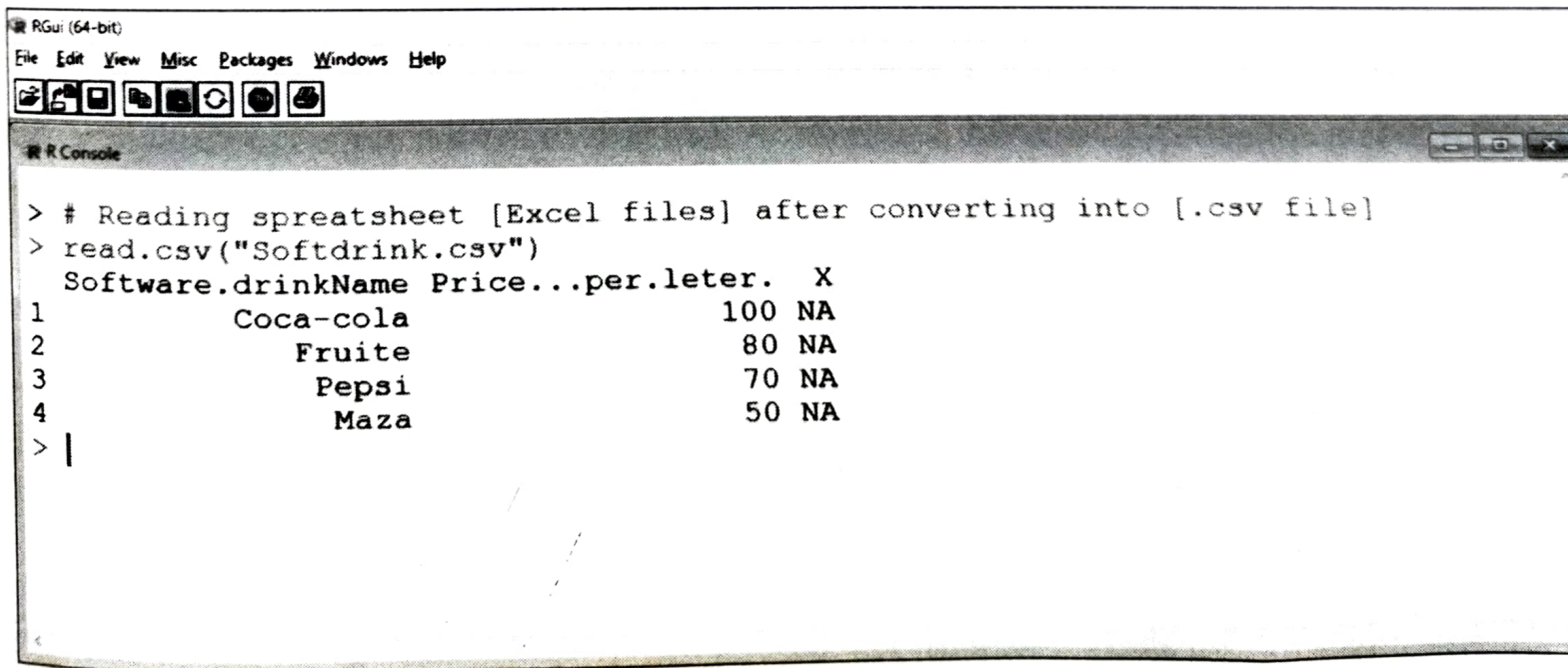
The following example illustrates creation of an Excel file, 'Softdrink.xlsx'. The 'Software.csv' file is the converted form of the 'Softdrink.xlsx' file (Figure 3.28). The function `read.csv()` is reading this file into R (Figure 3.29).



The screenshot shows the Microsoft Excel interface with the 'Softdrink.xlsx' file open. The spreadsheet has two columns: 'Software drinkName' and 'Price [ per leter]'. The data is as follows:

|   | A                  | B                  | C | D | E | F | G | H | I |
|---|--------------------|--------------------|---|---|---|---|---|---|---|
| 1 | Software drinkName | Price [ per leter] |   |   |   |   |   |   |   |
| 2 | Coca-cola          | 100                |   |   |   |   |   |   |   |
| 3 | Fruite             | 80                 |   |   |   |   |   |   |   |
| 4 | Pepsi              | 70                 |   |   |   |   |   |   |   |
| 5 | Maza               | 50                 |   |   |   |   |   |   |   |
| 6 |                    |                    |   |   |   |   |   |   |   |
| 7 |                    |                    |   |   |   |   |   |   |   |
| 8 |                    |                    |   |   |   |   |   |   |   |
| 9 |                    |                    |   |   |   |   |   |   |   |

FIGURE 3.28 Spreadsheet of Excel file



The screenshot shows the RGui interface with the R console open. The console output is as follows:

```
> # Reading spreatsheet [Excel files] after converting into [.csv file]
> read.csv("Softdrink.csv")
  Software.drinkName Price...per.leter. X
1          Coca-cola          100 NA
2           Fruite          80 NA
3           Pepsi          70 NA
4            Maza          50 NA
> |
```

### Example: Reading the .csv file

To read the data from a .csv file (D:\SampleSuperstore.csv) into a data frame. The data should be grouped by 'Category'. The column on which grouping is done is 'Sales'. The aggregate function to be used is 'sum'.

**Step 1:** The data is stored in 'D:\SampleSuperstore.csv'. It is available under the following columns:

Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, State, City, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, Discount, Price.

A subset of the data is shown in Figure 3.30.

With the use of read.csv function, the data is read from 'D:\SampleSuperstore.csv' file and stored in the data frame named, 'InputData'.

```
> InputData <- read.csv("d:/SampleSuperstore.csv")
```

**Step 2:** Data is grouped and aggregated on InputData\$Sales by InputData\$Category. The aggregation function used is 'sum'. InputData\$Sales refers to the 'Sales' column of the data frame, 'InputData'. Similarly, InputData\$Category refers to the 'Category' column of the data frame, 'InputData'.

```
> GroupedInputData <- aggregate(InputData$Sales ~
  InputData$Category, InputData, sum)
```

Display the aggregated data. As evident from the display below, the data is available in three categories, viz. 'Furniture', 'Office Supplies' and 'Technology'.

```
> GroupedInputData
  InputData$Category InputData$Sales
1           Furniture         156514.4
2   Office Supplies         132600.8
3           Technology         168638.0
```

### 3.13.2 Reading Data from Packages

A package is a collection of functions and datasets. In R, many packages are available for doing different types of operations (Figure 2.4). Some functions for reading and loading the dataset from and into packages defined in R are explained next.

#### library() Function

The library() function loads packages into the R workspace. It is compulsory to import the package before reading the available dataset of that package. The syntax of the library() function is:

```
library(packagename)
```

where,

packagename argument is the name of the package to be read.

### **3.13.3 Reading Data from Web/APIs**

Nowadays most business organisations are using the Internet and cloud services for storing data. This online dataset is directly accessible through packages and application Programming interfaces (APIs). Different packages are available in R for reading from online datasets. Refer to Table 3.6 to view some packages.



**TABLE 3.6** Packages for reading web data

| Packages              | Description                                                                                                    | Download Link                                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| RCurl                 | The package permits download of files from the web server and post forms.                                      | <a href="https://cran.r-project.org/web/packages/RCurl/index.html">https://cran.r-project.org/web/packages/RCurl/index.html</a>             |
| Google Prediction API | It allows uploading of data to Google storage and then training them for Google Prediction API.                | <a href="http://code.google.com/p/r-google-predictionapi-v121">http://code.google.com/p/r-google-predictionapi-v121</a>                     |
| Infochimps            | The package provides the functions for accessing all API.                                                      | <a href="http://api.infochimps.com">http://api.infochimps.com</a>                                                                           |
| HttpRequest           | The package reads the web data with the help of an HTTP request protocol and implements the GET, POST request. | <a href="https://cran.r-project.org/web/packages/httpRequest/index.html">https://cran.r-project.org/web/packages/httpRequest/index.html</a> |
| WDI                   | The package reads all World Bank data.                                                                         | <a href="http://cransprojectorg/web/packages/WDI/index.html">http://cransprojectorg/web/packages/WDI/index.html</a>                         |
| XML                   | The package reads and creates an XML and HTML document with the help of an HTTP or FTP protocol.               | <a href="http://cransprojectorg/web/packages/XML/index.html">http://cransprojectorg/web/packages/XML/index.html</a>                         |
| Quantmod              | The package reads finance data from Yahoo finance.                                                             | <a href="http://crans-projectorg/web/packages/quantmod/index.html">http://crans-projectorg/web/packages/quantmod/index.html</a>             |
| ScrapeR               | The package reads online data.                                                                                 | <a href="http://crans-projectorg/web/packages/scrapeR/index.html">http://crans-projectorg/web/packages/scrapeR/index.html</a>               |

The following example illustrates web scraping. Web scraping extracts data from any webpage of a website. Here package 'RCurl' is used for web scraping (Figure 3.32). At first, the package, 'RCurl' is imported into the workspace and then `getURL()` function of the package, 'RCurl' takes the required webpage. Now `htmlTreeParse()` function parses the content of the webpage.

### 3.13.4 Reading a JSON (Java Script Object Notation) Document

**Step 1: Install rjson package.**

```
> install.packages("rjson")
Installing package into 'C:/Users/seema_acharya/Documents/R/win-
library/3.2' (as 'lib' is unspecified)
trying URL 'https://cran.hafro.is/bin/windows/contrib/3.2/
rjson_0.2.15.zip'
Content type 'application/zip' length 493614 bytes (482 KB)
downloaded 482 KB

package 'rjson' successfully unpacked and MD5 sums checked
```

### 3.13.5 Reading an XML File

**Step 1: Install an XML package.**

```
> install.packages("XML")
Installing package into 'C:/Users/seema_acharya/Documents/R/win-
library/3.2' (as 'lib' is unspecified)
trying URL 'https://cran.hafro.is/bin/windows/contrib/3.2/XML_3.
1.3.zip'
Content type 'application/zip' length 4299803 bytes (4.1 MB)
downloaded 4.1 MB
```

package 'XML' successfully unpacked and MD5 sums checked

**Step 2: Input data.**

Store the data below in a text file (XMLFile.xml in the D: drive). Ensure that the file is saved with an extension of .xml.

```
<RECORDS>
  <EMPLOYEE>
    <EMPID>1001</EMPID>
    <EMPNAME>Merrilyn</EMPNAME>
    <SKILLS>MongoDB</SKILLS>
    <DEPT>Computer Science</DEPT>
  </EMPLOYEE>

  <EMPLOYEE>
    <EMPID>1002</EMPID>
    <EMPNAME>Ramya</EMPNAME>
    <SKILLS>People Management</SKILLS>
    <DEPT>Human Resources</DEPT>
  </EMPLOYEE>

  <EMPLOYEE>
    <EMPID>1003</EMPID>
    <EMPNAME>Fedora</EMPNAME>
    <SKILLS>Recruitment</SKILLS>
    <DEPT>Human Resources</DEPT>
  </EMPLOYEE>
</RECORDS>
```

### Reading an XML File

The xml file is read in R using the function `xmlParse()`. It is stored as a list in R.

## Check Your Understanding

1. What is a CSV file?

**Ans:** A CSV file uses .csv extension and stores data in a table structure format in any plain text.

2. What is the use of `read.csv()` function?

**Ans:** A `read.csv()` function reads data from CSV files.

3. What is the use of `read.table()` function?

**Ans:** A `read.table()` function reads data from text files or CSV files.

4. What is the use of `read.xlsx()` function?

**Ans:** A `read.xlsx()` is an inbuilt function of 'xlsx' package for reading Excel files.

5. What is a package?

**Ans:** A package is a collection of functions and datasets. In R, many packages are available for doing different types of operations.

6. What is the use of the `library()` function?

**Ans:** The `library()` function loads packages into the R workspace. It is compulsory to import packages before reading the available dataset of that package.

7. What is the use of `data()` function?

**Ans:** The `data()` function lists all the available datasets of the loaded packages into the workspace.

8. List five R packages for accessing web data.

**Ans:** Different packages are available in R for reading from an online dataset. These are

- RCurl
- Google Prediction API
- WDI
- XML
- ScrapeR

9. What is web scraping?

**Ans:** Web scraping extracts data from any web page of a website.



## 4.5 EXPLORING DATA

Data in R is a set of organised information. Statistical data type is more common in R, which is a set of observations where values for the variables are passed. These input variables are used in measuring, controlling or manipulating the results of a program. Each variable differs in size and type. R supports the following basic data types to explore:

- Integer
- Numeric
- Logical
- Character/string
- Factor
- Complex

Data types have been covered in Chapter 2. Based on the specific data characteristics in R, data can be explored in different ways. You will learn about these methods in the following section.

### 4.5.1 Exploratory Data Analysis

Exploratory data analysis (EDA) involves dataset analysis to summarise the main characteristics in the form of visual representations. Exploratory data analysis using R is an approach used to summarise and visualise the main characteristics of a dataset, which differs from initial data analysis. The main aim of EDA is to summarise and visualise the main characteristics of a dataset. It focuses on:

- Exploring data by understanding its structure and variables
- Developing an intuition about the dataset
- Considering how the dataset came into existence
- Deciding how to investigate by providing a formal statistical method
- Extending better insights about the dataset
- Formulating a hypothesis that leads to new data collection
- Handling any missing values
- Investigating with more formal statistical methods.



Some of the graphical techniques used by EDA are:

- Box plot
  - Histogram
  - Scatter plot
  - Run chart
  - Bar chart
  - Density plots
  - Pareto chart
-

**TABLE 4.2** Functions for obtaining data summary in R

| Function Name                  | Description                                        |
|--------------------------------|----------------------------------------------------|
| <code>summary(x)</code>        | Returns the min, max, median, and mean             |
| <code>min(x)</code>            | Returns the minimum value                          |
| <code>max(x)</code>            | Returns the maximum value                          |
| <code>range(x)</code>          | Returns the range of the given input               |
| <code>mean(x)</code>           | Returns the mean value                             |
| <code>median(x)</code>         | Returns the median value                           |
| <code>mad(x)</code>            | Returns the median absolute deviation value        |
| <code>IQR(x)</code>            | Returns the interquartile range                    |
| <code>quantile(x)</code>       | Returns quartiles                                  |
| <code>summary(x)</code>        | Summarises the data frame                          |
| <code>apply(x, 1, mean)</code> | Calculates the row mean value                      |
| <code>apply(x, 2, mean)</code> | Calculates the column mean which is similar to the |

We will execute few R functions on the data set, "Employee". Let the contents of the dataset, "Employee".

```
> Employee
      EmpNo  EmpName ProjName EmpExpYears
1    1000     Jack    P01           5
2    1001     Jane    P02           9
3    1002 Margaritta P03           6
4    1003        Joe    P04          12
5    1004        Dave    P05           7
```

The `summary(Employee[4])` function works on the fourth column and computes the minimum, 1<sup>st</sup> quartile, median, mean, 3<sup>rd</sup> quartile and its value.

```
> summary(Employee[4])
```